



PRODUCT DESCRIPTION GENERATOR

Graduation project
Team 1



OUR TEAM

Team Leader: Yasmine Mohamed Abdou

Ranim Yasser Mohamed

Hesham Ahmed Mohamed

Ahmed Basuony Elgendy

Omar Ashraf Mohamed





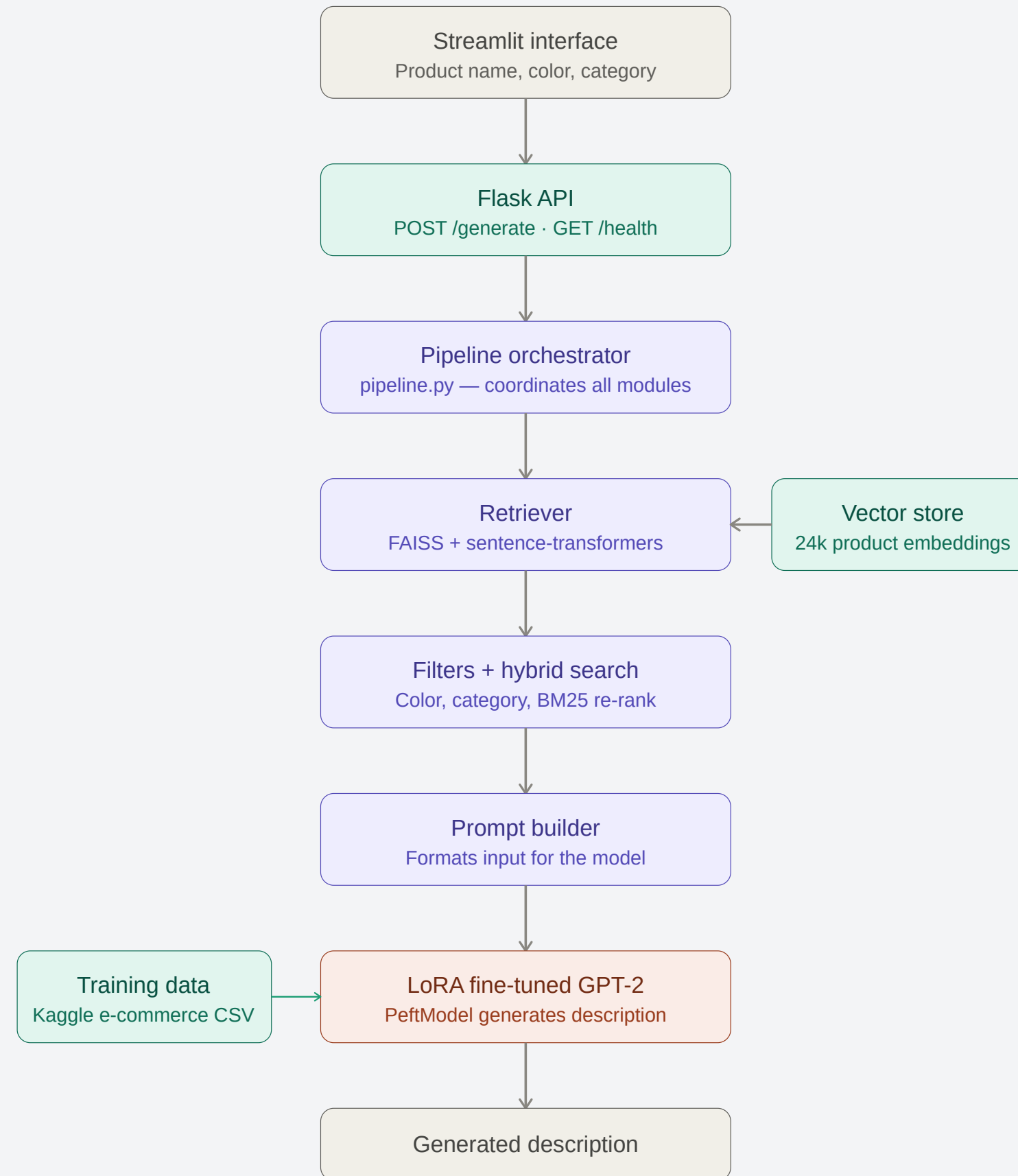
PROJECT DESCRIPTION

The project aims to build an AI-powered system that automatically generates high-quality product descriptions for e-commerce products. Using a dataset of real product listings, the system is trained to convert structured product information — such as product name, category, and key features — into marketing-ready descriptions.

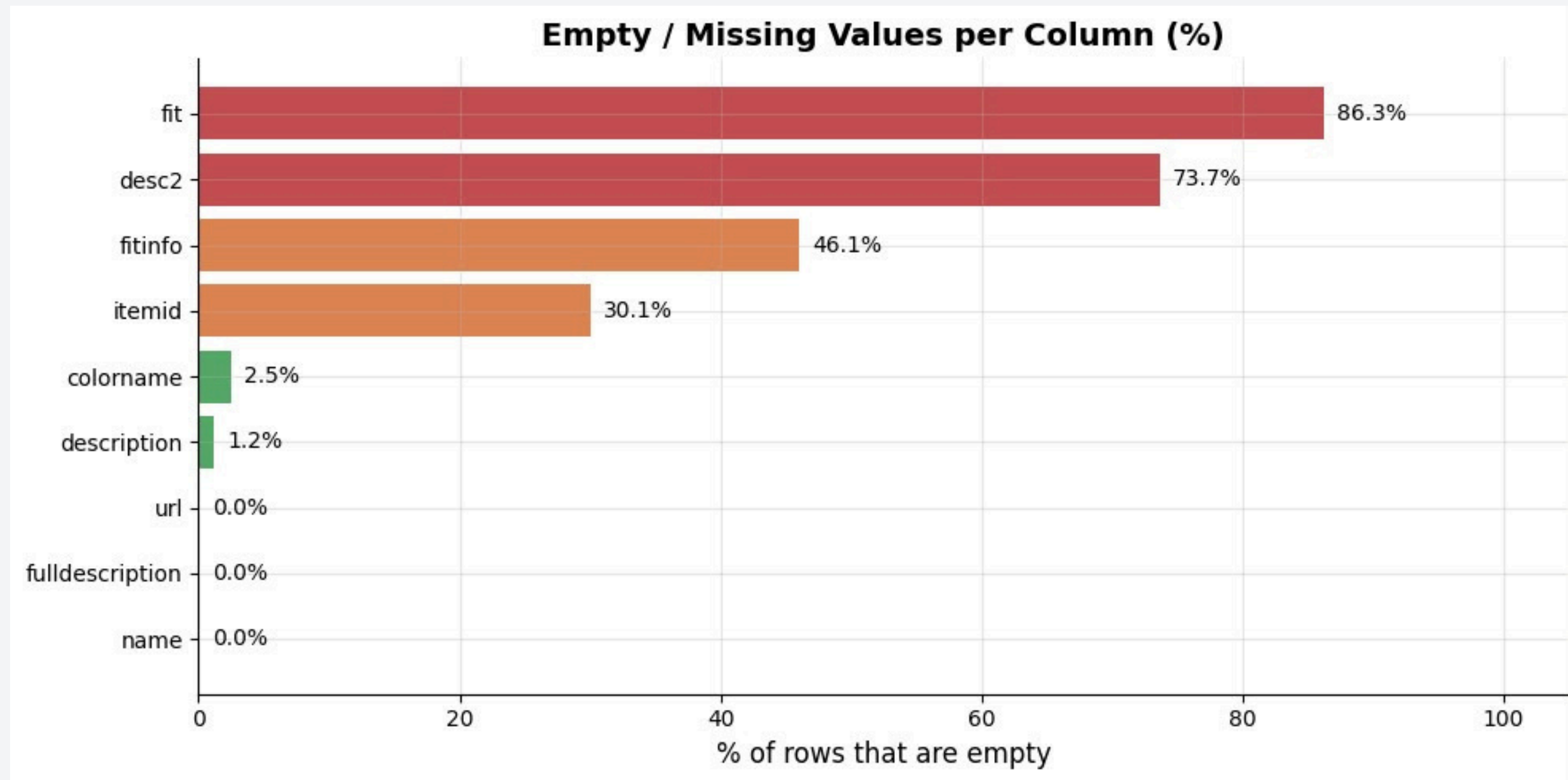
DATASET

- **dataset:** eCommerce data scraped from the L.L. Bean website
- **source:** kaggle
- **link:** <https://www.kaggle.com/datasets/mewbius/ecommerce-products?select=productsfull2.csv>
- **description:** contains 24k product names and their corresponding descriptions along side other features like color.

PROJECT PIPELINE

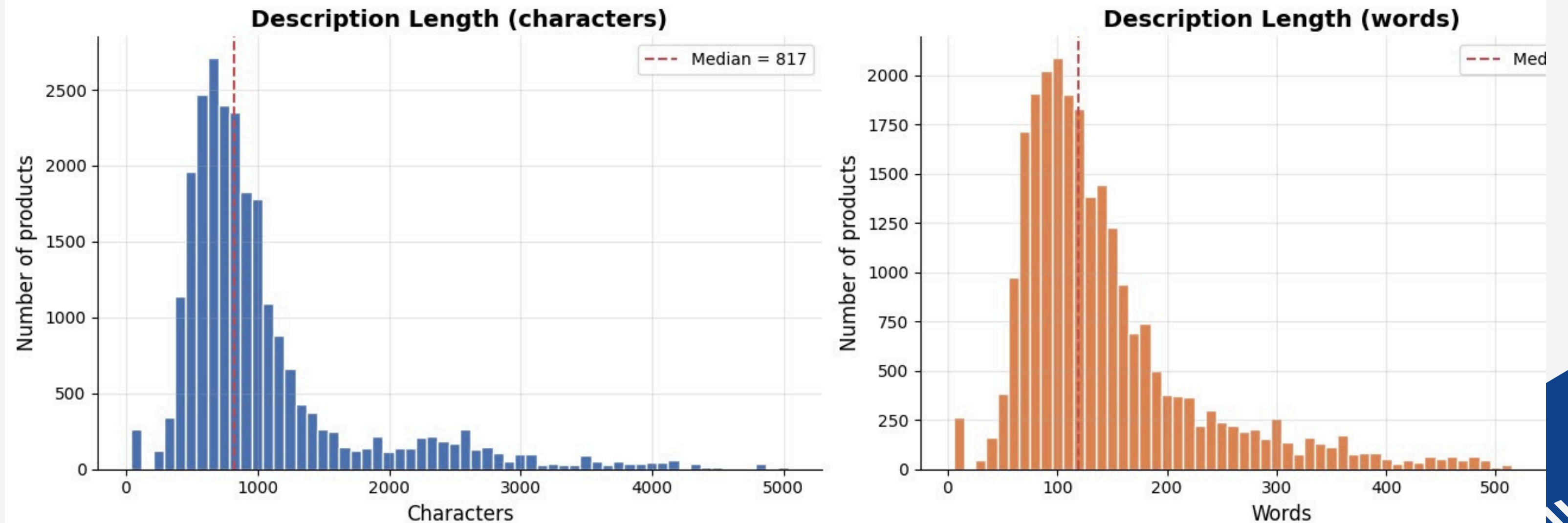


EXPLORATORY DATA ANALYSIS

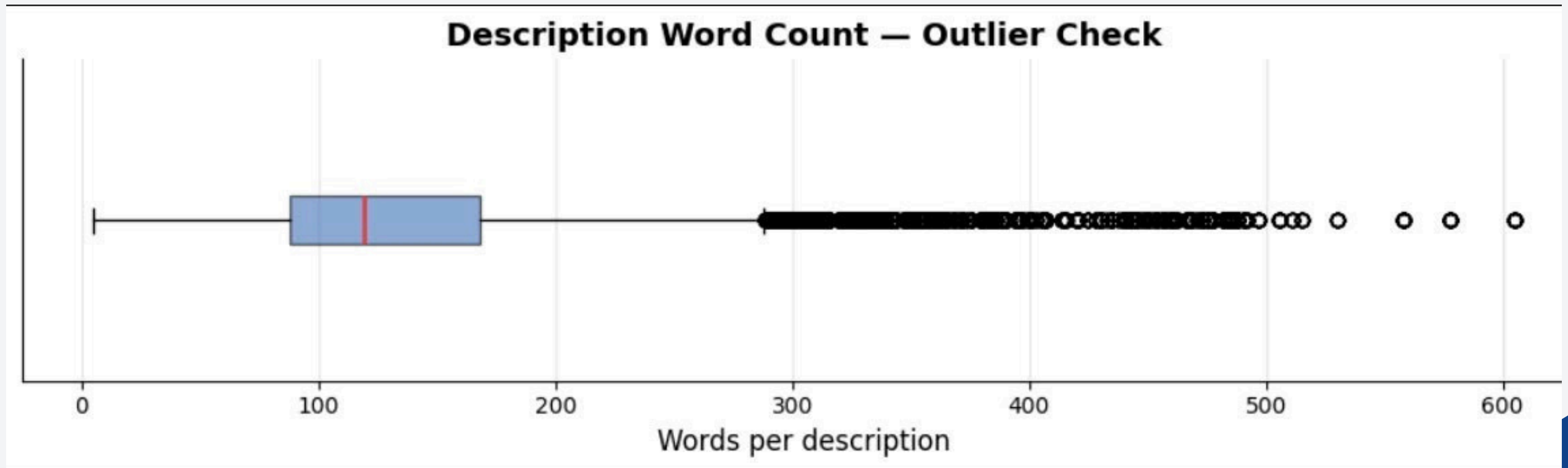


EXPLORATORY DATA ANALYSIS

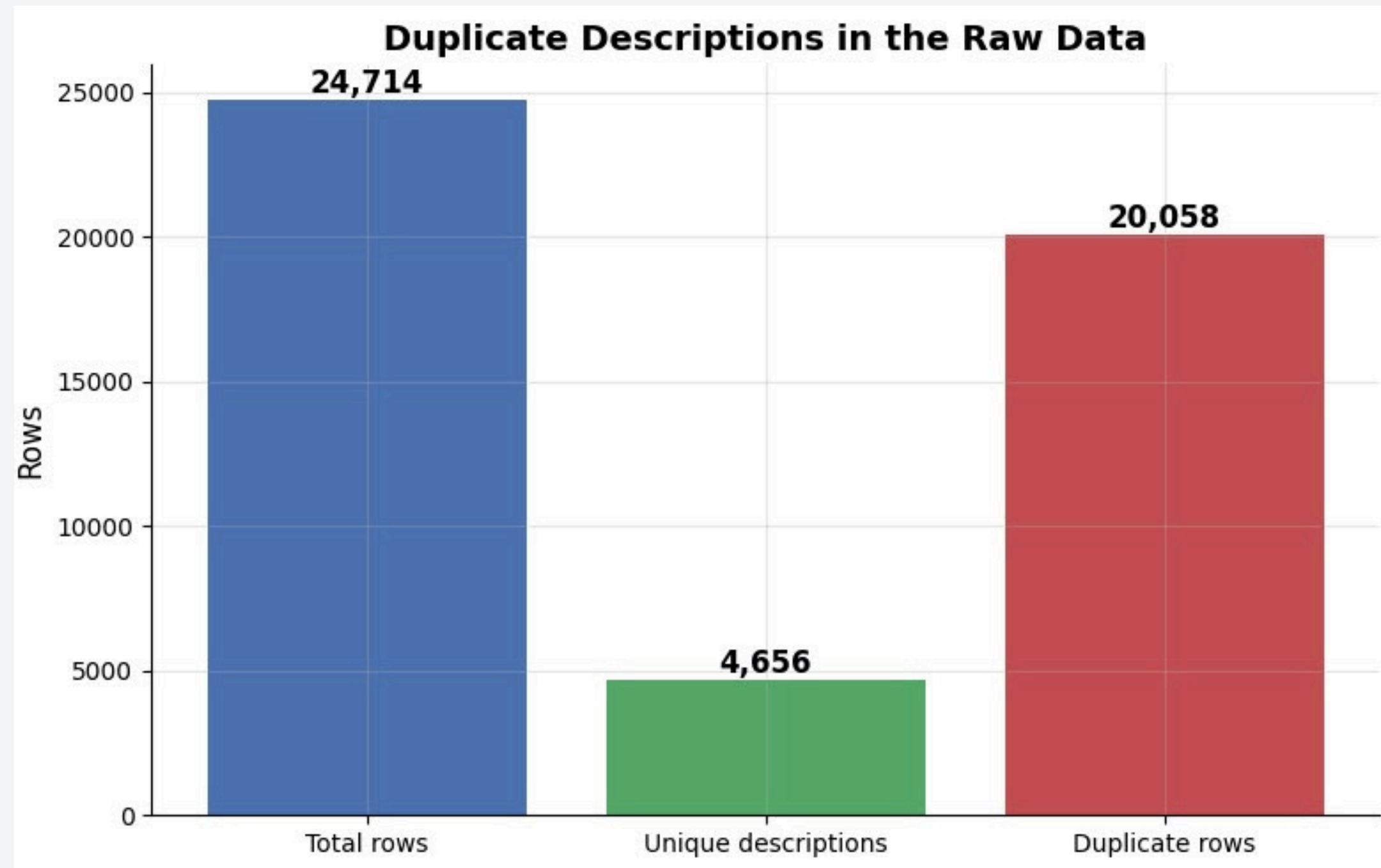
Raw Full-Description Length Distribution



EXPLORATORY DATA ANALYSIS



EXPLORATORY DATA ANALYSIS



RESULTS

EDA revealed:

- missing values
- empty strings written as []
- needs column selection
- average description length is between 300 to 500 words
- duplicates present

PREPROCESSING

- keep[name, fulldescription, colorname]
- replacing [] & dropping nan
- dropping short descriptions and duplicates
- text cleaning
- Saving the Schema into a clean CSV file



IMPLEMENTATION

RAG SYSTEM

RETRIEVAL-AUGMENTED GENERATION: GROUNDING THE MODEL IN REAL CATALOGUE DATA INSTEAD OF GENERATING BLIND.

DENSE RETRIEVAL

- SENTENCE-TRANSFORMERS (MINILM-L6) EMBEDS PRODUCT NAME + COLOR INTO 384-DIM VECTORS FOR FAISS COSINE-SIMILARITY SEARCH.

HYBRID SEARCH

- BM25 KEYWORD SCORES ARE MERGED WITH DENSE RESULTS AND DE-DUPLICATED, CATCHING EXACT-KEYWORD MATCHES THE EMBEDDING MODEL MIGHT UNDERRANK.

POST-RETRIEVAL FILTERS

- RESULTS ARE NARROWED BY COLOR AND CATEGORY, WITH AUTOMATIC FALLBACK TO THE UNFILTERED SET SO THE PIPELINE NEVER RETURNS EMPTY.

PROMPT ENRICHMENT

- RETRIEVED EXAMPLES ARE WOVEN INTO THE PROMPT AS FEW-SHOT CONTEXT FOR THE LORA-TUNED GPT-2 MODEL.

4,651

PRODUCTS INDEXED

384-DIM

EMBEDDING VECTORS

TOP-K

SIMILARITY RETRIEVAL



FILTERING & HYBRID SEARCH

FILTERING:

why? We don't rely solely on RAG because vector search is inherently fuzzy. This filtering layer acts as a hybrid bridge, combining the conceptual power of AI search with the strict accuracy of traditional database filtering.

1. filter by color
 2. filter by category
 3. all filters
- 

FILTERING & HYBRID SEARCH

HYBRID SEARCH:

Hybrid search: combines dense (FAISS) retrieval with BM25 keyword search. Improves results when the user's query shares exact keywords with products that the embedding model might rank lower.

1. creates a “corpus” to search through
2. dense retrieval for $k \times 2$ “over fetching”
3. works along side vector retrieval

PROMPT BUILDING

TRAINING FORMAT

PRODUCT: <NAME>

COLOR: <COLOR>

DESCRIPTION: <TEXT>

Built per-row from the cleaned dataset; prompt_length is tracked so overly long rows can be filtered before tokenization.

INFERENCE PROMPT

PRODUCT: <NAME>

COLOR: <COLOR>

DESCRIPTION:

Identical prefix, cut off right after "Description: " for the model to complete. The RAG-enhanced builder can prepend retrieved similar products as few-shot examples first.

SEQUENCE LENGTH ANALYSIS & MODEL SELECTION

- Analyzed raw (untruncated) prompt token lengths before choosing a cutoff
- Token lengths ranged 58–923 tokens per prompt
- Word-count distribution: mean 157 words, median 137, 75th percentile 185 words
- Chose max_length = 256 tokens for tokenization —
- GPT-2 has no native pad token → used EOS token as padding (tokenizer.pad_token = tokenizer.eos_token)

MODEL TRAINING & FINE-TUNNING

GPT-2 FINE-TUNED WITH LORA: ADAPTING A PRETRAINED MODEL WITHOUT TOUCHING ALMOST ANY OF ITS ORIGINAL WEIGHTS.

MODEL & LORA CONFIG

- BASE MODEL: GPT-2 (124M PARAMS)
- RANK $R = 8$, ALPHA = 16, DROPOUT = 0.1
- TARGET MODULES: C_ATTN (ATTENTION)
- TASK TYPE: CAUSAL LANGUAGE MODELING

TRAINING SETUP

- 3 EPOCHS, BATCH SIZE 2
- LEARNING RATE $2E-4$, 100 WARMUP STEPS
- MIXED PRECISION (FP16)
- 6,279 TOTAL OPTIMIZER STEPS

0.24%

OF PARAMS TRAINABLE
(294,912 / 124.7M)

5.48 → 2.34

TRAINING LOSS, STEP 50 → STEP 6,250

MODEL EVALUATION

FIVE COMPLEMENTARY METRICS ON 50 HELD-OUT
VALIDATION SAMPLES, GENERATED WITH THE SAME
DECODING SETTINGS USED AT INFERENCE.

1

PERPLEXITY

- HOW WELL THE MODEL PREDICTS HELD-OUT TEXT — LOWER IS BETTER.

2

BLEU-1...4

- N-GRAM OVERLAP BETWEEN GENERATED AND REFERENCE DESCRIPTIONS.

3

ROUGE-1/2/L

- RECALL-ORIENTED OVERLAP WITH THE REFERENCE TEXT.

4

DISTINCT-1/2

- UNIQUE UNIGRAM/BIGRAM RATIO — MEASURES OUTPUT DIVERSITY.

5

GEN. TIME / SAMPLE

- AVERAGE WALL-CLOCK TIME TO GENERATE ONE DESCRIPTION.

DECODING: TEMP 0.6 · TOP-K 40 · TOP-P 0.85 · REP. PENALTY 1.3 · NO-REPEAT 3-GRAMS



ORCHESTRATION

Connecting all modules together

Smart Resource Management:

- **Retrieve:** Fetches raw matches using semantic AI search.
- **Filter:** Enforces strict constraints (like matching color/category keywords).
- **Augment:** Packages the data into a structured prompt.
- **Generate:** Hands the prompt to the LLM to write the final response.

DEPLOYMENT

Framework: Powered by a Flask REST API hosted locally

Production Configuration: Configured to bind to 0.0.0.0 on port 7860

/health (GET Endpoint)

/generate (POST Endpoint): Accepts user inputs via structured JSON payloads (name, color, category, top_k) and feeds them to our pipeline.



THANK
YOU